

DEPARTAMENTO DE APLICACIONES MULTIPLATAFORMA

JavaScript



CICLO: 1 DAM

Alumno: Víctor Manuel Guadalupe Gonzalez

CURSO: 2025 / 2026

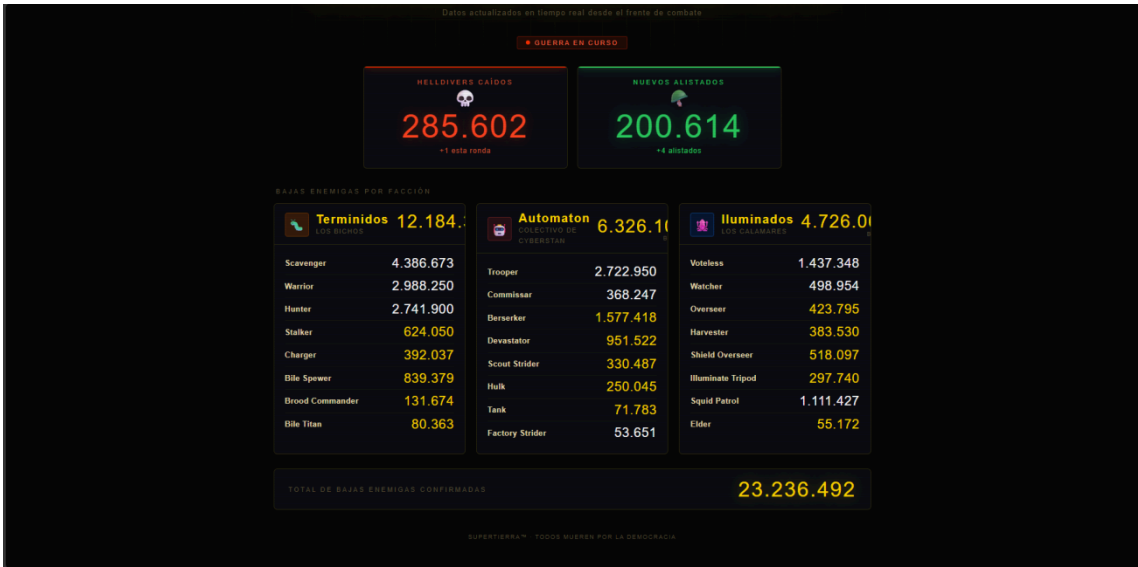
Índice

HTML Y IDEA	3
JavaScript y explicación	4
Validación de formularios	6
LINK A GITHUB:	9

HTML Y IDEA

En esta parte del proyecto de la página web he decidido basándome en una idea vista en internet donde en el contexto de mi página, siguiendo el rol de como si fuera un videojuego en el que hay una guerra constante, poner los marcadores simulando que son marcadores en vivo donde se indican distintos números que indican tanto pérdidas como la suma de nuevos aliados y enemigos.

Idea en la que me he basado



Mi versión simplificada



JavaScript y explicación

```
// Estado de los contadores
let stats = {
  fallen: 1000000,
  enlisted: 1200000,
  bugs: 2500000,
  squids: 5000000,
  automatons: 1200000
};
```

En el documento empiezo con let “**stats**” donde coloco los valores iniciales de cada “facción” por así llamarlos siendo obviamente números altos para representar lo máximo posible a los números reales.

```
// Función para obtener un número aleatorio en un rango
function randomInRange(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

El “**randomInRange**” es una función de utilidad pura con un “**Math.floor**” para que este entre un intervalo de números y no se salga de este siendo así que sea mas realista.

```
// Función para actualizar un contador
function updateCounter(elementId, deltaId, newValue, oldValue) {
  const element = document.getElementById(elementId);
  if (element) {
    element.textContent = newValue.toLocaleString('es-ES');
  }

  // Actualizar delta si existe
  if (deltaId) {
    const deltaElement = document.getElementById(deltaId);
    if (deltaElement) {
      const delta = newValue - oldValue;
      const symbol = delta > 0 ? '+' : '';
      deltaElement.textContent = symbol + delta.toLocaleString('es-ES');

      // Color según el valor
      if (delta > 0) {
        deltaElement.style.color = '■ #ff4444';
      } else if (delta < 0) {
        deltaElement.style.color = '■ #00ff88';
      } else {
        deltaElement.style.color = '■ #99b3cc';
      }
    }
  }
}
```

Aquí vemos un “**updateCounter**” que se dedica a actualizar el contador calculando el **delta** que es la diferencia entre el nuevo y el viejo valor y lo muestra con un + o sin simbolo cambiando de color verde o rojo segun su cambio.

Y claro tenemos el **deltaId** donde hacemos todas las operaciones y los colores que adoptará el contador en caso de los distintos cambios de valores

```
// Función para calcular y mostrar el total
function updateTotal() {
    const total = stats.fallen + stats.enlisted + stats.bugs + stats.squids + stats.automatons;
    const totalElement = document.getElementById('total-enemies');
    if (totalElement) {
        totalElement.textContent = total.toLocaleString('es-ES');
    }
}

// Actualizar contadores cada 2 segundos
setInterval(() => {
    // Helldivers caídos
    const oldFallen = stats.fallen;
    stats.fallen += randomInRange(5, 1000000);
    updateCounter('counter-fallen', 'delta-fallen', stats.fallen, oldFallen);

    // Nuevos alistados
    const oldEnlisted = stats.enlisted;
    stats.enlisted += randomInRange(3, 1000000);
    updateCounter('counter-enlisted', 'delta-enlisted', stats.enlisted, oldEnlisted);

    // Bichos abatidos
    const oldBugs = stats.bugs;
    stats.bugs += randomInRange(20, 6000000);
    updateCounter('counter-bugs', 'delta-bugs', stats.bugs, oldBugs);

    // Calamares abatidos
    const oldSquids = stats.squids;
    stats.squids += randomInRange(10, 4000000);
    updateCounter('counter-squids', 'delta-squids', stats.squids, oldSquids);

    // Autómatas abatidos
    const oldAutomatons = stats.automatons;
    stats.automatons += randomInRange(8, 3500000);
    updateCounter('counter-automatons', 'delta-automatons', stats.automatons, oldAutomatons);

    // Actualizar total
    updateTotal();
}, 2000);
```

Hacia el final vemos el **valor total** y como se va actualizando en relación de los números aleatorios en el intervalo en específico como en el caso de **"oldFallen"** con un intervalo entre 5 y 1000000 y su actualización de los números con **"updateCounter"**.

```
// Inicializar valores en la primera carga
window.addEventListener('DOMContentLoaded', () => {
  updateCounter('counter-fallen', 'delta-fallen', stats.fallen, stats.fallen);
  updateCounter('counter-enlisted', 'delta-enlisted', stats.enlisted, stats.enlisted);
  updateCounter('counter-bugs', 'delta-bugs', stats.bugs, stats.bugs);
  updateCounter('counter-squids', 'delta-squids', stats.squids, stats.squids);
  updateCounter('counter-automatons', 'delta-automatons', stats.automatons, stats.automatons);
  updateTotal();
});
```

Por ultimo **"El DOMContentLoaded"** garantiza que el HTML ya esté cargado antes de intentar tocar los elementos. Sin esto, el script podría ejecutarse antes de que existan los **id** en la página y no encontraría nada. Es la inicialización: pinta los valores de partida nada más abrir la página

Validación de formularios

Para las páginas de alistamiento y contacto he añadido dos ficheros JavaScript externos (**Alistate.js** y **contacto.js**) que validan los formularios antes de que el usuario los envíe, dando mensajes de error o de éxito según el resultado.

Alistate.js valida el formulario de alistamiento. Escucha el evento **submit** del formulario con **addEventListener** y llama a **e.preventDefault()** para evitar que la página se recargue.

A continuación comprueba campo a campo: que el nombre tenga al menos 3 caracteres, que el email tenga un formato correcto usando una expresión regular (**emailRegex**), que la edad sea mayor de 18, que se haya seleccionado una experiencia y una clase, y que la motivación tenga al menos 10 caracteres. Si alguna validación falla se muestra un mensaje de error en rojo; si todo está correcto aparece un mensaje de bienvenida en dorado con el nombre del usuario y el formulario se reinicia con **form.reset()**.

¡Bienvenido a las filas, Carlos Rodríguez González! La Supertierra te necesita.

ALISTAMIENTO HELLDIVERS

Nombre Completo:

asf

Correo Electrónico:

safa



Incluye un signo "@" en la dirección de correo electrónico. La dirección "safa" no incluye el signo "@".

60

¿Experiencia en combate?

☒ Principiante ☐ Intermedio ☐ Experto

Clase Preferida:

Asalto

¿Por qué quieres alistarte?

sa

ALISTARSE AHORA

¡Bienvenido a las filas, Carlos Rodríguez González! La Supertierra te necesita.

contacto.js sigue la misma estructura que el anterior. Valida que el nombre tenga al menos 3 caracteres, que el email sea válido, que el asunto tenga al menos 5 caracteres y que el mensaje tenga al menos 20 caracteres.

Los mensajes de error y de éxito se crean dinámicamente con **document.createElement** y se insertan al final del formulario, eliminando el anterior si ya existía gracias a la función **quitarMensaje()** para que no se acumulen. Los colores de los mensajes respetan el estilo visual de la página.

Formulario de Contacto

Nombre:

Email:

Asunto:

Mensaje:

474

Enviar

El mensaje debe tener al menos 20 caracteres.

¡Mensaje enviado! El cuartel general te responderá en breve.

LINK A GITHUB:

https://github.com/victinix30k/ActividadFinal_PaginaWeb_JavaScript